

Playtesting the Immortal Engine

Where AI helps, where it can't, and how you should play

A field guide to AI-assisted playtesting, the state of the art, its hard limits, and the part that will always be yours.

Prepared for Tim · Immortal Engine · 2026-06-27

The short version

AI playtesting has quietly matured. In the last two years it has gone from a research curiosity to an **assembly of proven patterns** that find the “is it broken?” layer of a game **more reliably than human testers do** — crashes, dead-ends, soft-locks, and cross-turn incoherence, across hundreds of paths a person would never have the patience to walk.

But there is a hard ceiling. Every one of these systems certifies that a game is **coherent**. None of them can tell you it is **good**. They answer “does this work?” They are blind to “is this magic?”

*So AI doesn't replace your playtesting. It **relocates** it. The machine takes over the machine's job — hunting broken plumbing — so your scarce attention goes to the only job it can't do: judging whether the game is worth playing.*

And the Immortal Engine has a structural advantage almost no game has: because the deterministic core owns its **Canon Log** (the authoritative record of what actually happened), the single highest-value test — checking whether the words match the world — is essentially free. Most of the research below spends its effort on a problem you simply don't have.

Part 1 · The state of the art in AI playtesting

Two separate research literatures matured to the point where an automated “play the game and find the bugs” harness is now an **assembly problem**, not an invention:

- **Automated game-playing QA agents** (the RL era 2018–22, the LLM era 2024–26) reliably out-explore human testers and surface crash, reachability, stuck, and logic bugs no author would think to script.
- **Long-form coherence evaluation** (2025–26) now has a formal *taxonomy of incoherence* and an automated checker that beats human experts at catching it.

Across both, the winning design is the same four things:

- **A bounded action set** the agent chooses from — not free-form generation. (This is the cure for “action-space paralysis,” where an unconstrained agent emits nonsense and stalls.)
- **A bank of cheap oracles**, each watching for one failure family: crash, stuck, slow, contradiction.
- **A structured, evidence-bearing report** — a quote plus a location plus a class — not a single score.
- **A ground-truth substrate** the prose or behavior is checked against. The strongest systems bolt this on. The Immortal Engine already *is* this (the Canon Log).

The systems worth knowing

These are the load-bearing results — the ones that prove the pattern works and give us numbers.

System (year)	What it is	The result that matters
TITAN (2025)	An LLM agent that tests live commercial MMORPGs. Four oracles (crash, stuck-task, slow, logic) plus “reflective stuck-detection”: after ~20 no-progress actions it diagnoses the impasse and re-strategizes.	82% of bugs found vs ~45% for prior bots; 73% state coverage vs 34% for a human tester. Caught a quest that completed after 5 kills when it required 6 — the classic “logic” bug.
Lap (2025)	A deliberately thin perceive–prompt–act loop on a match-3 game, with only coverage + crash oracles.	79% coverage vs 3–5% for a random bot. Proof that even a minimal loop crushes scripted/random testing — the floor is low to clear.
Jin et al. (ACL 2024)	Bug detection in an <i>LLM-powered text game</i> — our exact class. Reads game logs and flags hallucination, forgetfulness, and prompt-misinterpretation.	Structured log-analysis beats unstructured LLM bug-hunting. The cheapest possible start: re-judge transcripts you already have.
ConStory (2026)	A taxonomy of narrative incoherence (5 categories, 19 subtypes) plus an automated checker that extracts suspect spans, pairs them, and builds an evidence chain.	F1 0.678 vs human experts' 0.229 at catching injected consistency bugs. Automated coherence checking is not a lossy shortcut — it catches <i>more</i> than reading by hand.
Reachability line (Go-Explore, EA SEED, CTL)	Exploration agents that prove every intended area is reachable and find places a player gets stuck or can't escape.	A large share of game bugs are reachability bugs. “Can the player actually finish?” is a first-class, well-studied failure family.

What AI playtesting can reliably do

- **Crash and stability** — the oldest, most solid use.
- **Soft-lock / reachability** — “I’m stuck,” “this quest can’t be finished,” “there’s no way out of this room” — found by a goal-directed agent that notices it isn’t making progress.
- **Coherence / desync** — the words contradicting the world, the NPC who forgets, the count that’s off by one, the fact that flips between turns.
- **Regression** — proving a change didn’t silently break something that used to work.
- **Saturation measurement** — running across hundreds of seeds and watching the new-bugs-per-100 curve flatten, so “we found roughly all of them” becomes a *number*, not a *vibe*. (This is the real answer to “so I don’t have to playtest forever.”)

How this maps onto the Immortal Engine

You are not starting from scratch — most of the apparatus exists, and the hardest research module is free for you:

- **The deterministic desync oracle (the crown jewel).** After every turn, diff what the narration *claims* against what the Canon Log *committed*. “You step outside” — did the interior actually clear? “Rook falls” — is the foe actually dead? This catches the entire movement / state-desync class, runs in CI at **zero AI cost**, and has none of the judge-reliability risk. It exists only because you own the Canon Log.
- **The 10 deterministic bug classes** (`scripts/playtest.js`): crash, invariant violation, determinism break, save corruption, ending leak, and more — run across thousands of turns per sweep.
- **The experiential gate** (`scripts/dm-playtest.mjs`): adversarial personas + an LLM judge over the live DM, used as a *discovery pointer* for qualitative failures.
- **What the research says to add** (per the project’s own Vol 16 survey): a goal-directed player carrying a typed objective that selects from the legal-action set; a soft-lock oracle; and a coherence judge that checks prose *against the Canon Log* — turning fuzzy “is this self-consistent?” questions into precise “does the prose match state?” checks.

*The one fact that changes everything for this game: every paper spends most of its machinery on **perception** — turning pixels into something an agent can reason about. You are already text plus a deterministic state log. The hardest part of the literature is free for you.*

Part 2 · What AI cannot replace

This is the part to internalize, because the failure mode is seductive: a green harness *feels* like a finished game. It isn't. Here is the honest boundary.

The fundamental limit: coherent is not the same as good

Every oracle above certifies that the game holds together. Not one of them can tell you it is worth playing. Prose that sings, a world that feels alive, pacing, surprise, whether a new player even realizes they *can* search the room — the harness is blind to all of it. It certifies **coherent**, never **delightful**.

Wire the harness to gate the word “playable.” Never let it gate the word “fun.” Fun only ever comes out of human hands on the keyboard.

The proof from your own project: Phase B

This isn't theory. The talk->quest->consequence loop — the spine of the whole game — was marked **“complete and proven”**: it closed end-to-end, deterministically, and every residual the gate surfaced was fixed. And yet the project's own ledger records the limit in plain language:

“The loop's wow wants a Tim taste-read (a real playthrough); the adversarial gate can't supply it.”

“Proven” meant the machine confirmed the loop is not broken. Whether it *lands* — whether a stranger feels the small magic of being greeted by name in a town they've never visited — is a judgment only a person at the keyboard can make. The automated gate didn't even walk the loop; it measured robustness around it.

Why you can't just point more AI at it

- **An AI judge is not generator-invariant.** A model judging output from its own family inflates the score (self-preference). The project's rule — *no Opus-judging-Opus* — exists for this reason. Even the best coherence checker sits at F1 ~0.68: good enough to *point*, not to *rule*.
- **Metrics get gamed.** Optimize to any single judge and you drift toward what the judge rewards, not what is good. A high gate score with a halo effect can mask a dull game.
- **The number bounces.** The experiential gate is persona-sampled and stochastic — its headline % moves run to run. Treat a swing as noise to investigate, never as a goalpost to chase.
- **AI can't set the vision.** It optimizes a metric you give it. It cannot decide what the game *should* be — the north star, the taste, the line between “weird-good” and “weird-broken.” That is the founder's job, permanently.

The judgments that are yours alone

The machine can tell you...	Only you can tell you...
The game didn't crash	The game was fun
The quest can be completed	The quest was worth completing

The machine can tell you...	Only you can tell you...
The words match the world state	The words made you feel something
The NPC didn't forget a fact	The NPC felt like a person
No path leaves the player stuck	The pacing pulled you forward
Nothing regressed since last build	Something surprised or delighted you
The loop closes, deterministically	You wanted to keep playing

Part 3 · How you should playtest, as a human

The whole point of the AI layer is to **buy back your attention**. Don't spend a minute of human playtime doing what an oracle can do. Spend all of it on the ceiling the machine can't reach.

1. Let the three signals do their jobs — and keep them separate

The project already runs on a three-signal model. The discipline is to never let one impersonate another:

Signal	What it is	What it certifies
Corpus	The deterministic test suite + invariants + the desync/soft-lock oracles. Runs every change.	Regression — nothing drifted; the world still holds together.
Gate	The adversarial personas + LLM judge over the live DM. Run periodically.	Discovery — a noisy pointer to qualitative failures worth a look.
Human (you)	Your hands on the real game, live.	Taste — the only verdict on whether it's good. Nothing else can supply it.

The trap to avoid (and the one the project already fell into once): claiming something is “proven” on the corpus/gate axis while the human axis was never run. Coherent is not good. Let the machine gate “playable”; reserve “fun” for your own hands.

2. Play at the rung-climb moments

You don't need to play constantly — that's what the harness is for. You need to play at the **exact moment the machine says “functional / proven.”** That is precisely when the felt-quality gap appears (it's what happened at Phase B). When a capability lands and the corpus is green, that's your cue to sit down and ask the questions no oracle can.

3. Play the real surface, the way a player does

The project's own playtest protocol, distilled — it applies to you as much as to any agent:

- **Play the live game** (v1.html), not a test harness or an API. The engine runs in the browser; a passing server test can prove nothing about what you actually see.
- **Trust the screen, not the logs.** Real failures here were invisible-but-present: narration sat in the page but was clipped behind the map. If you didn't see it work, it didn't work.
- **Type what a real player types** — the messy, off-script, “can I befriend the dragon” inputs — not the happy path.
- **Keep one honest note** on what you didn't reach, instead of assuming it works.

4. Bring the questions only you can answer

This is the actual work of human playtesting. Sit down with the game and interrogate the *feeling*:

- Did I **want to keep playing** — or was I done and just being polite?
- Did anything **surprise or delight** me? Did the world ever feel **alive** rather than merely consistent?
- Did the **pacing** pull me forward, or did I stall, bored, between beats?
- As a **new player**, did I understand what I could even do? Did the game invite the move, or did I have to know the trick?

- Did the DM do **what a real DM would do** — resolve my intent in the fiction — or bounce it back as a mechanical prompt? (This is the project's governing taste test: the DM Test and the Table Test.)
- Did a moment land the way it was *meant* to — the small wow — or merely happen without error?

5. Use the machine to make “every problem” a number

When you (or a worker) aim the harness at a slice of the game — say, the first building everyone passes through — the move is: run a goal-directed player across hundreds of seeds, watch the discovery curve flatten, then fix by **seam** (the ~3–5 systemic causes), not by symptom. Each fix hardens every location, and each failing seed becomes a standing regression test. Saturate, fix the seam, advance. That converts “is it broken?” into a closed, measurable question — and frees you to spend your time on “is it good?”

The one-line takeaway

*Tests prove the logic. The harness proves the game is **coherent**. Only you, at the keyboard, can prove it's **magic** — so spend the machine's tirelessness on the first two, and spend yourself entirely on the third.*

Further reading inside the repo: docs/HARNESS_USAGE_STRATEGY.md (how to aim it), docs/PLAYTEST_PROTOCOL.md (hand over a working game), docs/biblioteca/vol-16-automated-playtesting-coherence-harness.md (the full research survey), and docs/THE_DM_TEST.md / THE_TABLE_TEST.md (the taste tests no oracle can run).